

Apache Kafka

Aurreratua

Kafka klusterra, AEMET Bronze/Silver/Gold pipeline-a, NiFi integrazioa eta Kafka Connect.

Zikloa Adimen Artifiziala eta Big Data

Modulua Big Data Aplikatua



01

2. Kasua — Kafka Klusterra

Kafka Kluster baten topologia

- **NODO 1 + BROKER 1** — garapena, probak, ariketa sinpleak
- **NODO 1 + BROKER ANITZ** — kluster baten portaera simulatzeko
- **NODO ANITZ + BROKER ANITZ** — produkzio-inguruneetan ohikoa

Docker Compose - 3 brokerreko klusterra

Broker 1
localhost:9092

Broker 2
localhost:9093

Broker 3
localhost:9094

Broker bakoitzak:

- Broker bakoitzak **ID propioa** dauka
- Broker bakoitzak **bere biltegitratzea** dauka
- Hirurak **Kafka kluster bereko kideak** dira
- Konfigurazioa **compose.yaml** fitxategian definituko da

— 3 BROKERREKO KLUSTERRA

Docker Compose – compose.yaml

```
compose.yaml
1  services:
2
3  # =====
4  # KAFKA BROKER 1
5  # =====
6  kafka1:
7    image: apache/kafka:latest
8    container_name: iabd-kafka1
9    hostname: kafka1
10
11    ports:
12      - "9092:9092"
13
14    environment:
15      KAFKA_NODE_ID: 1
16      KAFKA_PROCESS_ROLES: broker,controller
17
18      # KRaft Controller Quorum
19      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
20      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
21
22      # Listeners
23      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
24      KAFKA_LISTENERS: INTERNAL://:19092,CONTROLLER://:9093,EXTERNAL://:9092
25      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka1:19092,EXTERNAL://localhost:9092
26      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
27
28      # KRaft Cluster ID
29      CLUSTER_ID: 4L6g3nShT-eMctK--X86sw
30
31      # Kafka barneko topic-en erreplikazioa
32      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
33      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
34      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
35
36      # Kafka komandoak zuzenean erabiltzeko
37      PATH: "/opt/kafka/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

```
40  # =====
41  # KAFKA BROKER 2
42  # =====
43  kafka2:
44    image: apache/kafka:latest
45    container_name: iabd-kafka2
46    hostname: kafka2
47
48    ports:
49      - "9093:9092"
50
51    environment:
52      KAFKA_NODE_ID: 2
53      KAFKA_PROCESS_ROLES: broker,controller
54
55      # KRaft Controller Quorum
56      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
57      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
58
59      # Listeners
60      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
61      KAFKA_LISTENERS: INTERNAL://:19092,CONTROLLER://:9093,EXTERNAL://:9092
62      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka2:19092,EXTERNAL://localhost:9093
63      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
64
65      # KRaft Cluster ID
66      CLUSTER_ID: 4L6g3nShT-eMctK--X86sw
67
68      # Kafka barneko topic-en erreplikazioa
69      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
70      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
71      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
72
73      # Kafka komandoak zuzenean erabiltzeko
74      PATH: "/opt/kafka/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

```
77  # =====
78  # KAFKA BROKER 3
79  # =====
80  kafka3:
81    image: apache/kafka:latest
82    container_name: iabd-kafka3
83    hostname: kafka3
84
85    ports:
86      - "9094:9092"
87
88    environment:
89      KAFKA_NODE_ID: 3
90      KAFKA_PROCESS_ROLES: broker,controller
91
92      # KRaft Controller Quorum
93      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
94      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
95
96      # Listeners
97      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
98      KAFKA_LISTENERS: INTERNAL://:19092,CONTROLLER://:9093,EXTERNAL://:9092
99      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka3:19092,EXTERNAL://localhost:9094
100      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
101
102      # KRaft Cluster ID
103      CLUSTER_ID: 4L6g3nShT-eMctK--X86sw
104
105      # Kafka barneko topic-en erreplikazioa
106      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
107      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
108      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
109
110      # Kafka komandoak zuzenean erabiltzeko
111      PATH: "/ont/kafka/bin:/usr/local/sbin:/usr/local/bin:/usr/shin:/usr/bin:/shin:/bin"
```

Docker Compose – compose.yaml

```
114 # =====
115 # APACHE NIFI
116 # =====
117 nifi:
118   image: apache/nifi:latest
119   container_name: iabd-nifi-kafka
120   hostname: nifi
121
122   ports:
123     - "8443:8443"
124
125   environment:
126     SINGLE_USER_CREDENTIALS_USERNAME: nifi
127     SINGLE_USER_CREDENTIALS_PASSWORD: nifinifinifi
128
129   volumes:
130     - ./irteera:/opt/nifi/kafka-output
131
132   depends_on:
133     - kafka1
134     - kafka2
135     - kafka3
```

Kafka Listeners- Nondik konektatzen gara?

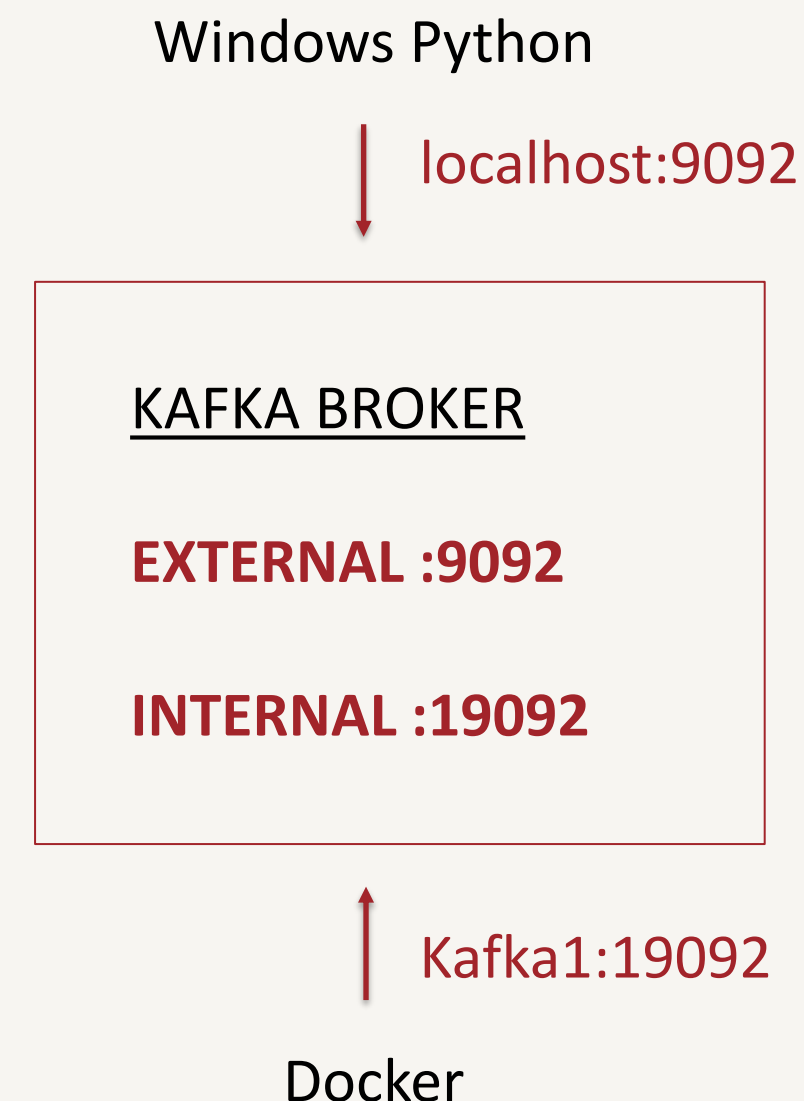
Listener batek Kafka broker-era nola eta nondik konektatu definitzen du.
Kafka broker batek listene bat baino gehiago izan ditzake.

Gure klusterrean 2 listener mota interesatzen zaizkigu:

- **EXTERNAL (9092)** → Docker-etik **KANPOKO** bezeroentzat.
Adib.: Python Windows-en.
- **INTERNAL (19092)** → Docker sare **BARRUKO** komunikaziorako.
Adib.: NiFi eta Kafka broker-ak.

ADI!

Bootstrap-server aukeratzeko, begiratu komandoa NON exekutatzen ari garen



Topic-a sortzeko adib.: 3 partizio + 2 replication-factor

```
●●● topic-3p2r.sh bash

1 # Topic-a sortu - 3 partizio eta 2 errepikatze-faktore
2 kafka-topics.sh --create --topic iabd-topic-3p2r \
3   --bootstrap-server kafka1:19092 \
4   --partitions 3 \
5   --replication-factor 2
6
7 # Topic-a deskribatu
8 kafka-topics.sh --describe --topic iabd-topic-3p2r \
9   --bootstrap-server kafka1:19092
10
11 # OHARRAK:
12 #   --partitions 3          -> topic-a 3 partiziotan banatu
13 #   --replication-factor 2  -> partizio bakoitzaren 2 kopia
14 #   --bootstrap-server     -> klusterrarekin lehen konexioa egiteko bróker bat
15 #   3 broker-eko klusterrean RF=2 -> 3 broker behar lukete
16 #                               (RF <= broker-kopurua araua)
17 #   Partizio bakoitzak Leader 1 + Follower 1 izango ditu.
```

Topic-a deskribatu

●●● describe-3p2r.txt

text

```
1 Topic: iabd-topic-3p2r PartitionCount: 3 ReplicationFactor: 2
2
3 Topic: iabd-topic-3p2r Partition: 0 Leader: 1 Replicas: 1,2 Isr: 1,2
4 Topic: iabd-topic-3p2r Partition: 1 Leader: 2 Replicas: 2,3 Isr: 2,3
5 Topic: iabd-topic-3p2r Partition: 2 Leader: 3 Replicas: 3,1 Isr: 3,1
6
7 # INTERPRETAZIOA:
8 #   3 partizio x 2 kopia = 6 replika guztira
9 #   Broker bakoitzak 2 replika ditu (6/3=2)
10 #
11 #   Partizio 0: Leader: Broker 1, Follower: Broker 2
12 #   Partizio 1: Leader: Broker 2, Follower: Broker 3
13 #   Partizio 2: Leader: Broker 3, Follower: Broker 1
14 #
15 #   Leader-ak broker desberdinen artean banatuta daude (lan-zama orekatua)
16 #   ISR = une honetan sinkronizatuta dauden erreplikak
17 #   ISR = Replicas -> erreplika guztiak sinkronizatuta
```

Producer-a klusterrera konektatu

●●● producer-cluster.py

python

```
1 from kafka import KafkaProducer
2 from json import dumps
3 import time
4
5 # Producer-a HIRU broker-en bootstrap-listarekin
6 producer = KafkaProducer(
7     value_serializer=lambda v: json.dumps(v).encode('utf-8'),
8
9     bootstrap_servers=[
10         'localhost:9092',    # broker 1
11         'localhost:9093',    # broker 2
12         'localhost:9094'])   # broker 3
13
14 # 10 mezu round-robin partition-banaketa (gakorik gabe)
15 for i in range(10):
16     producer.send("iabd-topic-3p2r",
17         value={"izena": "ekoizlea " + str(i)})
18
19 time.sleep(1)    # buffer-bidaltzeko denbora
20 producer.flush() # ziurtatu mezu guztiak bidalita
```

Consumer-a klusterrera konektatu

●●● consumer-cluster.py

python

```
1 from kafka import KafkaConsumer
2 from json import loads
3
4 consumer = KafkaConsumer(
5     'iabd-topic-3p2r',
6     auto_offset_reset='earliest',
7     enable_auto_commit=True,
8     group_id='iabd-taldea-1',
9     value_deserializer=lambda v: json.loads(v.decode('utf-8')),
10    bootstrap_servers=[
11        'localhost:9092',
12        'localhost:9093',
13        'localhost:9094'])
14
15 # Partizio, offset eta gakoa erakutsi
16 for m in consumer:
17     print(f"P:{m.partition} O:{m.offset} K:{m.key} V:{m.value}")
```

Producer-ak key-rik erabiltzen ez badu:

●●● output-no-key.txt

text

```
1 P:1 O:0 K:None V:{'izena': 'ekoizlea 0'}
2 P:1 O:1 K:None V:{'izena': 'ekoizlea 3'}
3 P:2 O:0 K:None V:{'izena': 'ekoizlea 1'}
4 P:2 O:1 K:None V:{'izena': 'ekoizlea 5'}
5 P:0 O:0 K:None V:{'izena': 'ekoizlea 2'}
6 P:0 O:1 K:None V:{'izena': 'ekoizlea 4'}
7
8 # INTERPRETAZIOA:
9 #   K: None -> mezuak ez du key-rik
10 #   Mezuak partizio desberdinetan egon daitezke
11 #   Offset-a PARTIZIO BAKOITZAREN barruan independentea da:
12 #   Partition 0 -> O:0, O:1...
13 #   Partition 1 -> O:0, O:1...
14 #   Partition 2 -> O:0, O:1...
15 #   Ez dago ordena globalik
16 #   PARTIZIO BAKOITZAREN BARRUAN ordena mantentzen da
```

Key bera → mezu guztiak partizio berera

... producer-key.py

python

```
1 # Producer-aren SEND gakoarekin
2 producer.send("iabd-topic-3p2r",
3     value={"izena": "ekoizlea " + str(i)},
4     key=b"iabd") # gako bera mezu guztietan
5
6 # CONSUMER-EKO IRTEERAREN ADIBIDEA:
7 # P:0 O:3 K:b'iabd' V:{'izena': 'ekoizlea 0'}
8 # P:0 O:4 K:b'iabd' V:{'izena': 'ekoizlea 1'}
9 # P:0 O:5 K:b'iabd' V:{'izena': 'ekoizlea 2'}
10 # P:0 O:6 K:b'iabd' V:{'izena': 'ekoizlea 3'}
11 # ...
12 # P:0 O:12 K:b'iabd' V:{'izena': 'ekoizlea 9'}
13
14 # INTERPRETAZIOA:
15 # Key bera -> partizio bera
16 # Partizio zehatza (P0, P1 edo P2) producer-ak zehazten du.
17 # Offset-ak gorantz doaz partizio horren barruan
18 # Kafka-k partizio baten barruko mezuen ordena mantentzen du
```

Kafka Klusterra – ideia nagusiak

- Kluster batek **broker anitz** izan ditzake
- Topic-ak **partiziotan** banatzen dira
- Partizio bakoitzak **Leader** bat dauka
- **Replication Factor**-ak partizio bakoitzaren kopia kopurua adierazten du
- **ISR** -> sinkronizatuta dauden erreplikak
- Producer/Consumer-ek **bootstrap_servers** bidez klusterra aurkitzen dute
- **Key** bera -> **partizio bera** -> **ordena mantendu**

2. KASUA: Kafka Klusterra + Python + NiFi

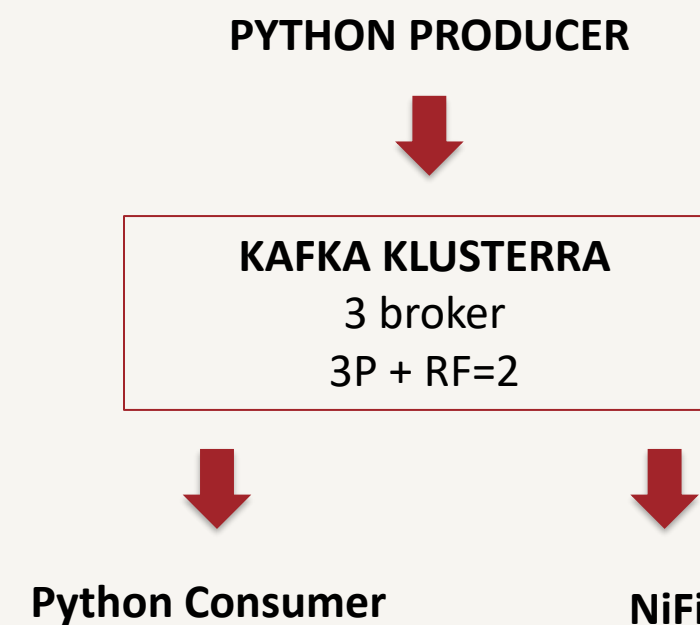
EGOERA

Sortu 3 brokerreko Kafka kluster bat Docker Compose erabiliz.

Sortu topic bat:

- 3 partizio
- Replication Factor = 2

Ondoren, Kafka topic bera hiru tresnarekin erabiliko dugu:



2. KASUA: Kafka Klusterra + Python + NiFi

EGINKIZUNAK

1. Kafka klusterra

- Sortu **compose.yaml** 3 brokerrekin eta NiFi-rekin eta abiarazi.

2. Topic-a

- Sortu topic bat: **3 partizio + RF=2** **ADI! Zein Bootstrap-server erabili?**

Komando hau Kafka container-aren barruan exekutatzen ari gara.

Beraz, Docker sare barruko listener-a erabili behar dugu: **kafka1:19092**

- Aztertu **--describe** erabiliz: **Leader, Replicas** eta **ISR**

3. Python Producer

- Faker erabiliz, sortu **10 pertsona**
- Bidali pertsona bakoitza Kafka mezu batean
- Datuak: *izena, adina, hiria, emaila*

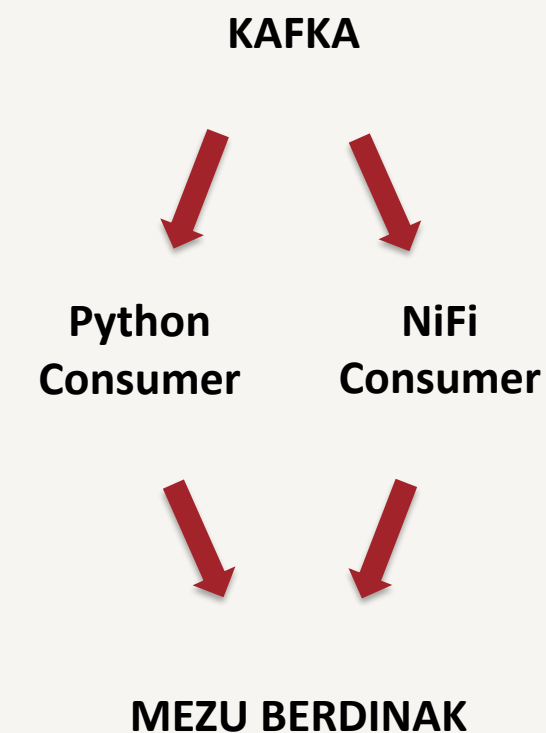
4. Python Consumer

- Kontsumitu topic-a eta erakutsi: **Partition | Offset | Key | Value**

5. NiFi Consumer

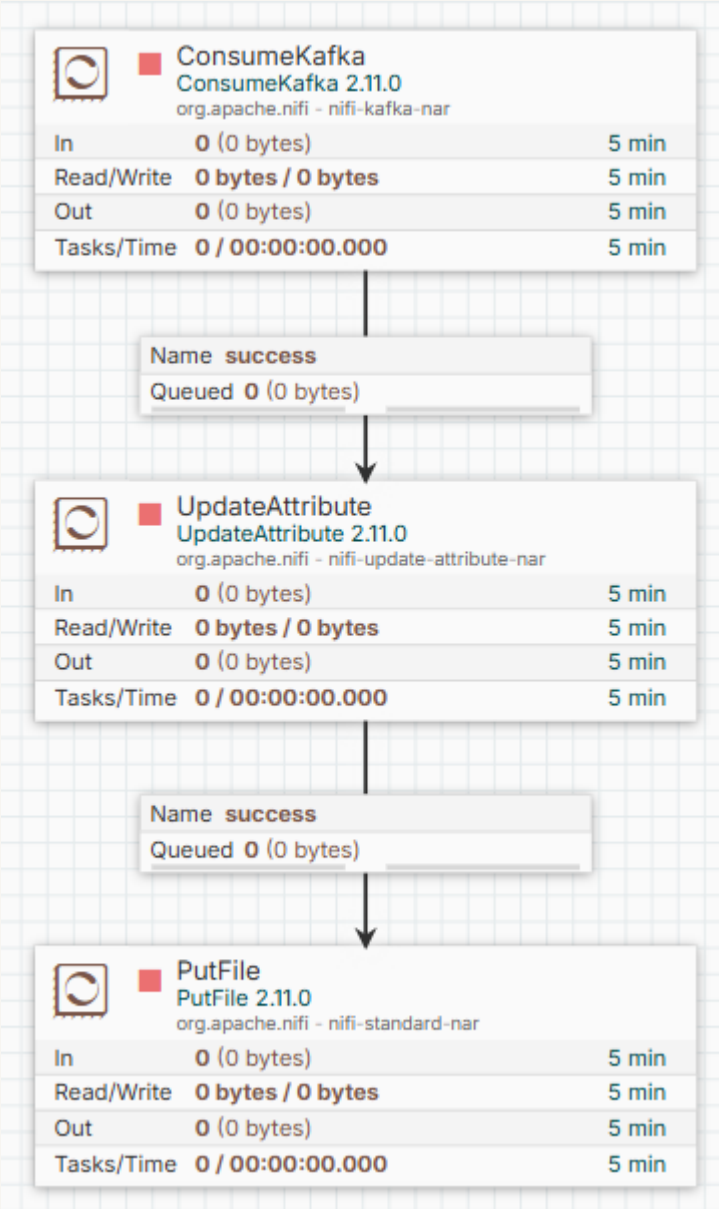
- Kontsumitu **topic bera** NiFi-tik (*ConsumeKafka* prozesadorea)
- Erabili Python Consumer-aren **consumer group desberdina**

6. Egiaztatu:



2. KASUA: Kafka Klusterra + Python + NiFi

NIFI-ko fluxua:



02

3. Kasua — AEMET

Bronze/Silver/Gold

— ZER EGINGO DUGU?

AEMET – Bronze / Silver / Gold

HELBURUA: eguraldi-datuak hainbat fasetan prozesatzea, jeruza bakoitza Kafka topic baten bidez konektatuz.

- **Bronze** – datu gordinak
- **Silver** – datu garbi eta egituratuak
- **Gold** – datu agregatuak / KPlak

Datuak pixkanaka prozesatu eta analisirako prestatuko dira.

Eguraldi API-a



BRONZE



SILVER



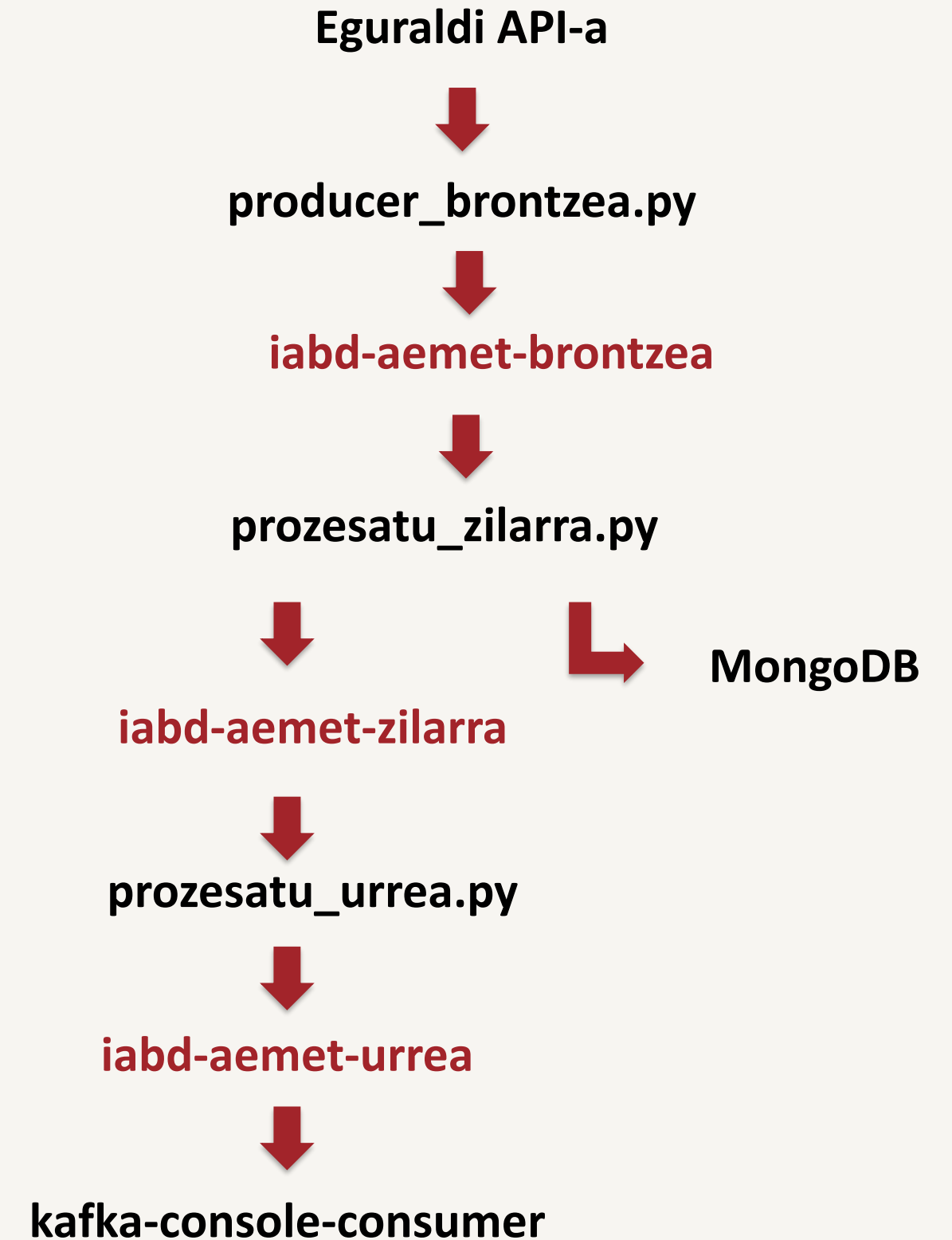
GOLD

AEMET – Bronze / Silver / Gold

Pipeline-a script independentez osatuta dago.

Pentsatu: script bakoitza zer da?

- Producer?
- Consumer?
- Consumer + Producer?



Hiru topic-ak sortu (Bronze, Silver, Gold)

Aurreko kasuan erabilitako Kafka klusterra erabili.

Sortu hiru topic:

- **iabd-aemet-brontzea**
- **iabd-aemet-zilarra**
- **iabd-aemet-urrea**

Topic-ak sortzeko: **kafka-topics.sh --create ...**

Egiaztatzeko: **kafka-topics.sh --list ...**

Bronze producer-a sortu: producer_brontzea.py

Programak honako hau egin behar du:

1. Eguraldi API-ari eskaera egin.
 - APIa: <https://api.el-tiempo.net/json/v3/provincias/03/municipios/03065>
2. JSON erantzuna jaso.
3. JSON osoa **iabd-aemet-brontzea** topic-era bidali.
4. 10 segundo itxaron.
5. Prozesua etengabe errepikatu.

Pistak: `requests.get(...)`
`KafkaProducer(...)`
`producer.send(...)`
`time.sleep(...)`

Zer espero dugu Bronze-n?

API-ak JSON nahiko handia itzuliko du.

Adibidez:

```
{
  "municipio": {
    "NOMBRE": "Elche/Elx",
  },
  "temperatura_actual": "21",
  "humedad": "55",
  ...
}
```

BRONZE-n ez dugu:

- eremurik kentzen
- datu-motarik aldatzen
- agregaziorik egiten

Datu gordina gordetzen dugu.

Bronze egiaztatzeko:

- bronze topic-a kontsumitu terminal batetik:
`kafka-console-consumer.sh --topic iabd-aemet-brontzea --from-beginning ...`

Sortu prozesatu_zilarra.py

prozesatu_zilarra.py programak honako hau egin behar du:

1. **iabd-aemet-brontzea** topic-a kontsumitu
2. JSON-etik beharrezko eremuak atera.
3. Silver objektu berria sortu.
4. **iabd-aemet-zilarra** topic-era bidali.
5. Silver datua MongoDB-n gorde.

Silver egitura

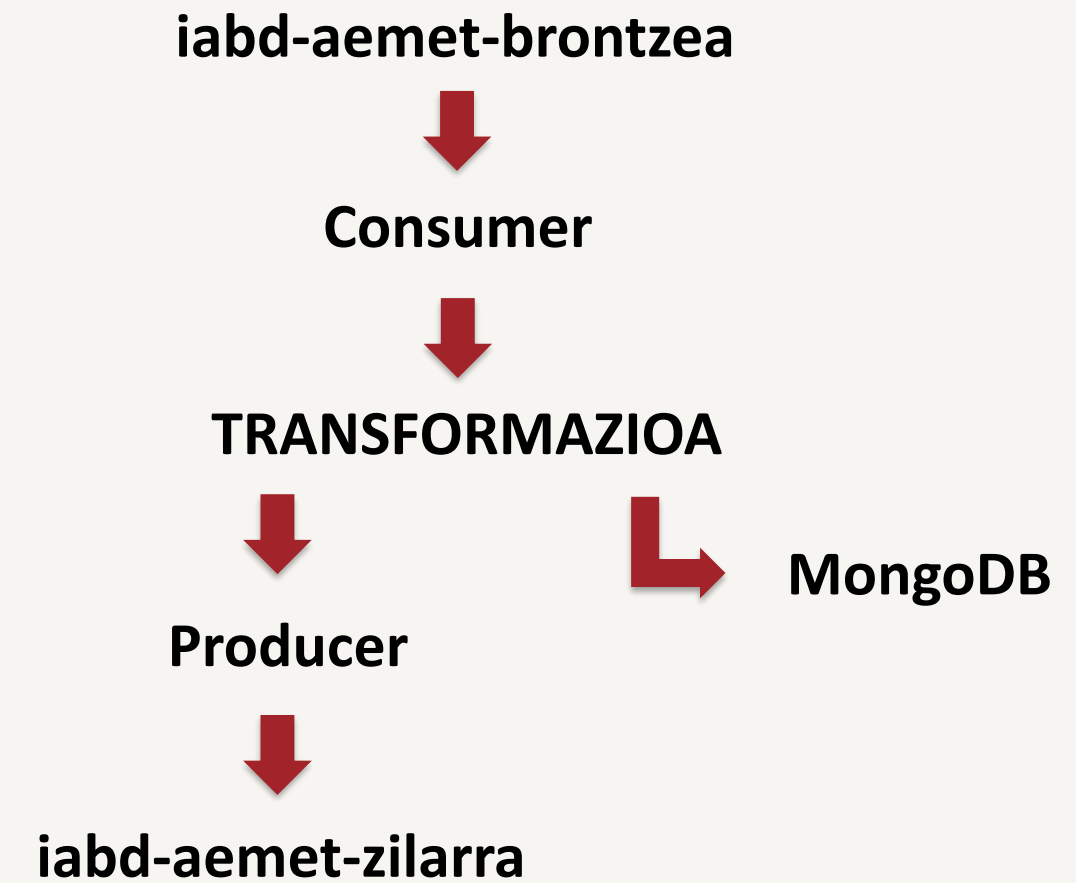
Silver objektuak egitura hau izan behar du: {

```
{
  "data": "...",
  "hiria": "...",
  "tenperatura": 0.0,
  "hezetasuna": 0.0
}
```

Baldintzak:

- **data:** uneko data eta ordua
- **hiria:** API-ko udalerriaren izena
- **tenperatura:** float
- **hezetasuna:** float

Pistak: `KafkaConsumer(...)`
`Json.loads(...)`
`Datetime.now(...)`
`float(...)`
`KafkaProducer(...)`
`MongoClient(...)`



Pentsatu: Consumer-a da? Producer-a da? Biak?

Silver egiaztatzeko:

- KAFKA: silver topic-a kontsumitu terminal batetik:
`kafka-console-consumer.sh --topic iabd-aemet-silver...`
- MONGODB: `use iabd`
`db.aemet_zilarra.find()`

Sortu prozesatu_urrea.py

prozesatu_urrea.py programak honako hau egin behar du:

1. **iabd-aemet-zilarra** topic-a kontsumitu.
2. Silver mezuak zerrenda batean Bildu.
3. 10 neurketa jasotakoan Pandas DataFrame bat sortu.
4. Estatistikak kalkulatu:
 - Batez_besteko_temperatura
 - Temperatura_min
 - Temperatura_max
 - Batez_besteko_hezetasuna
5. Gold objektua sortu.
6. **iabd-aemet-urrea** topic-era bidali.
7. Hurrengo 10 neurketekin berriro hasi.

Pistak:

- Pandas inportatu: `import pandas as pd`
- Silver mezuak zerrenda batean bildu: `mezuak = []`
- 10 mezu bildu diren egiaztatu: `if len(mezuak) == 10:`
- Zerrenda Pandas DataFrame bihurtu: `df = pd.DataFrame(mezuak)`
- Estatistikak kalkulatu:
 - Batezbestekoa: `.mean()`
 - Min: `.min()`
 - Max: `.max()`

Kontuan izan:

- 10 neurketa → Gold mezu 1

Zer espero dugu Gold-en?

10 neurketetatik kalkulatutako emaitza.

Adibidez:

```
{
  "hiria": "Elche/Elx",
  "neurketak": 10,
  "batez_besteko_tenperatura": 20.8,
  "tenperatura_min": 19.0,
  "tenperatura_max": 22.0,
  "batez_besteko_hezetasuna": 57.4
}
```

Gold egiaztatzeko:

- gold topic-a kontsumitu terminal batetik:
`kafka-console-consumer.sh --topic iabd-aemet-urrea --from-beginning ...`

Egiaztatu:

- Hasieran ez da Gold mezurik berehala agertzen.
- 10 Silver neurketa bildu behar dira.
- Ondoren Gold agregazioa agertu behar da.

Pipeline osoa martxan jarri

Ireki hainbat terminal.

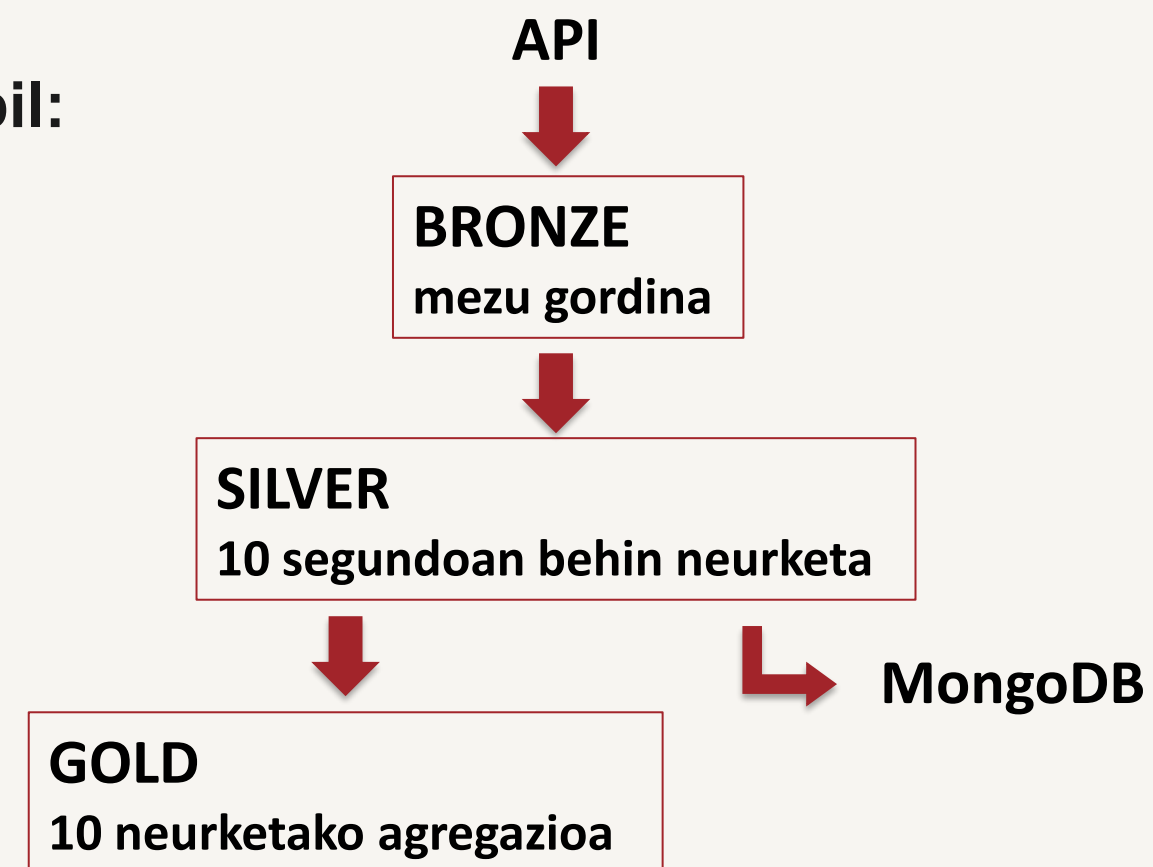
Exekuzio-ordena gomendatua:

1. Gold prozesua: `prozesatu_urrea.py`
2. Silver prozesua: `prozesatu_zilarra.py`
3. Bronze producer-a: `producer_brontzea.py`

Zergatik ordena honetan?

Consumer-ak prest egongo dira Producer-a datuak bidaltzen hasi baino lehen.

Dena ondo badabil:



Egiaztatu:

- Bronze-topic-ak mezuak ditu.
- Silver topic-ak mezuak ditu.
- MongoDB-k Silver dokumentuak ditu.
- Gold topic-ak agregazioak ditu.

— CONSUMER GROUP

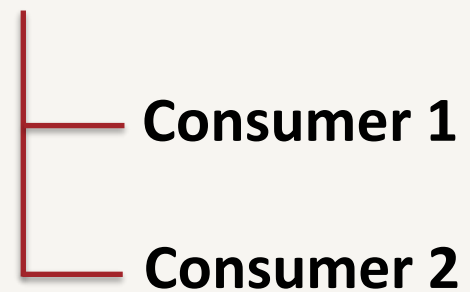
Consumer Group-ak berrikusi

Gogoratu:

GROUP ID BERA:

Consumer group bereko consumer-ek lana banatzen dute.

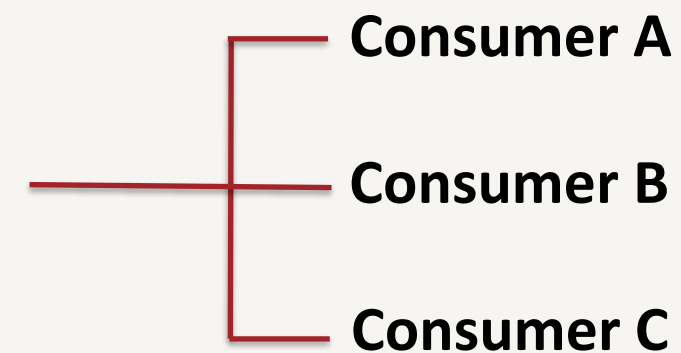
Topic



GROUP ID DESBERDINA:

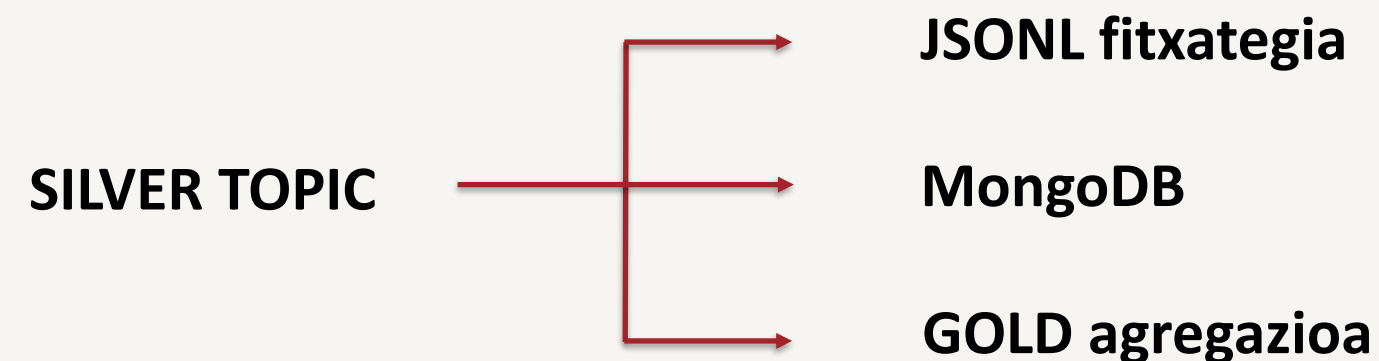
Consumer group bakoitzak mezu guztiak modu independentean irakur ditzake.

Topic



3. Kasua — Silver kontsumitzailea banatu

Orain Silver topic-eko datuak hiru helburu independente izango ditu:



Sortu hiru script independente:

- kontsumitzaileZilarraFitxategia.py
- kontsumitzaileZilarraMongoDB.py
- kontsumitzaileZilarra_EkoizleaUrrea.py

1. consumer independentea: kontsumitzaileZilarraFitxategia.py

Programak egin behar duena:

- **iabd-aemet-zilarra** kontsumitu.
- Mezu bakoitza fitxategi batean gorde:
 - Fitxategia: **silver_datuak.jsonl**
 - Mezu bakoitzak lerro bat okupatu behar du.

Consumer Group: **iabd-3kasua-fitxategia**

Pista: `open(..., "a")`
`json.dumps(...)`

2. consumer independentea: kontsumitzaileZilarraMongoDB.py

Programak egin behar duena:

- **iabd-aemet-zilarra** kontsumitu.
- Mezu bakoitza MongoDB-n gorde:

Consumer Group: **iabd-3kasua-mongo**

Egiaztatu: `db.aemet_zilarra.find(...)`

3. consumer independentea: kontsumitzaileZilarra_EkoizleaUrrea.py

Programak egin behar duena:

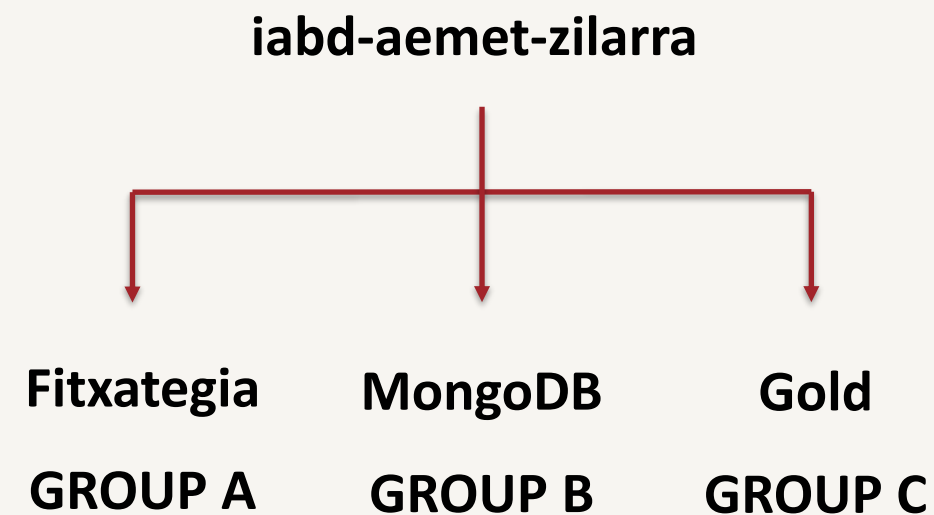
- **iabd-aemet-zilarra** kontsumitu.
- 10 mezu bildu.
- Pandas erabiliz agregazioa egin.
- Eraitza **iabd-aemet-urrea** topic-era bidali.

Consumer Group: **iabd-3kasua-gold**

Zergatik group ID desberdinak?

Hiru script-ek Silver mezu **guztiak** jaso behar dituzte.

Beraz:



Galdera: Zer gertatuko litzateke hirurek hau erabiliko balute? `group_id="iabd-3kasua"`

Pentsatu consumer group-en funtzionamenduan.

Erronka egiaztatu

Hiru script-ak aldi berean exekutatu.

Egiaztatu:

- **JSONL:** `silver_datuak.jsonl` datuak jasotzen ari da.
- **MongoDB:** `db.aemet_zilarra.find()` dokumentuak agertzen dira.
- **Gold:** Gold topic-ean agregazioak agertzen dira.

Consumer group-ak: `kafka-consumer-groups.sh --list --bootstrap-server...`

Hiru group ID desberdin agertu behar dira.

Azken hausnarketa

Erantzun:

1. Zer desberdintasun dago Bronze, Silver eta Gold artean?
2. Zergatik erabiltzen dugu topic bat geruza bakoitzaren artean?
3. **prozesatu_zilarra.py** Producer-a da ala Consumer-a da?
4. Zer abantaila du Bronze datuak gordetzeak?
5. Zer gertatzen da consumer bat gelditzen bada eta berriro abiarazten bada?
6. Zertarako erabiltzen da **group_id**?
7. Zergatik behar dituzte azken erronkako hiru consumer-ek Group ID desberdinak?
8. Zein abantaila du prozesua script independenteetan banatzeak?

Zer ikasi dugu?

API → Kafka → Python → Kafka → Python → Kafka

- Kafka topic-ak pipeline-etapak konektatzeko erabili ditugu.
- Producer eta Consumer-ak konbinatu ditugu.
- Bronze → Silver → Gold eredua aplikatu dugu.
- Python-ekin datuak eraldatu ditugu.
- Pandas-ekin agregazioak egin ditugu.
- MongoDB-n Silver datuak gorde ditugu.
- Consumer group-en portaera praktikan aplikatu dugu.
- Topic bera hainbat helburutarako kontsumitu dugu.

Kafka-k zerbitzu independenteak datu-fluxu baten bidex konektatzeko aukera ematen du.

03

4. Kasua — NiFi + Kafka

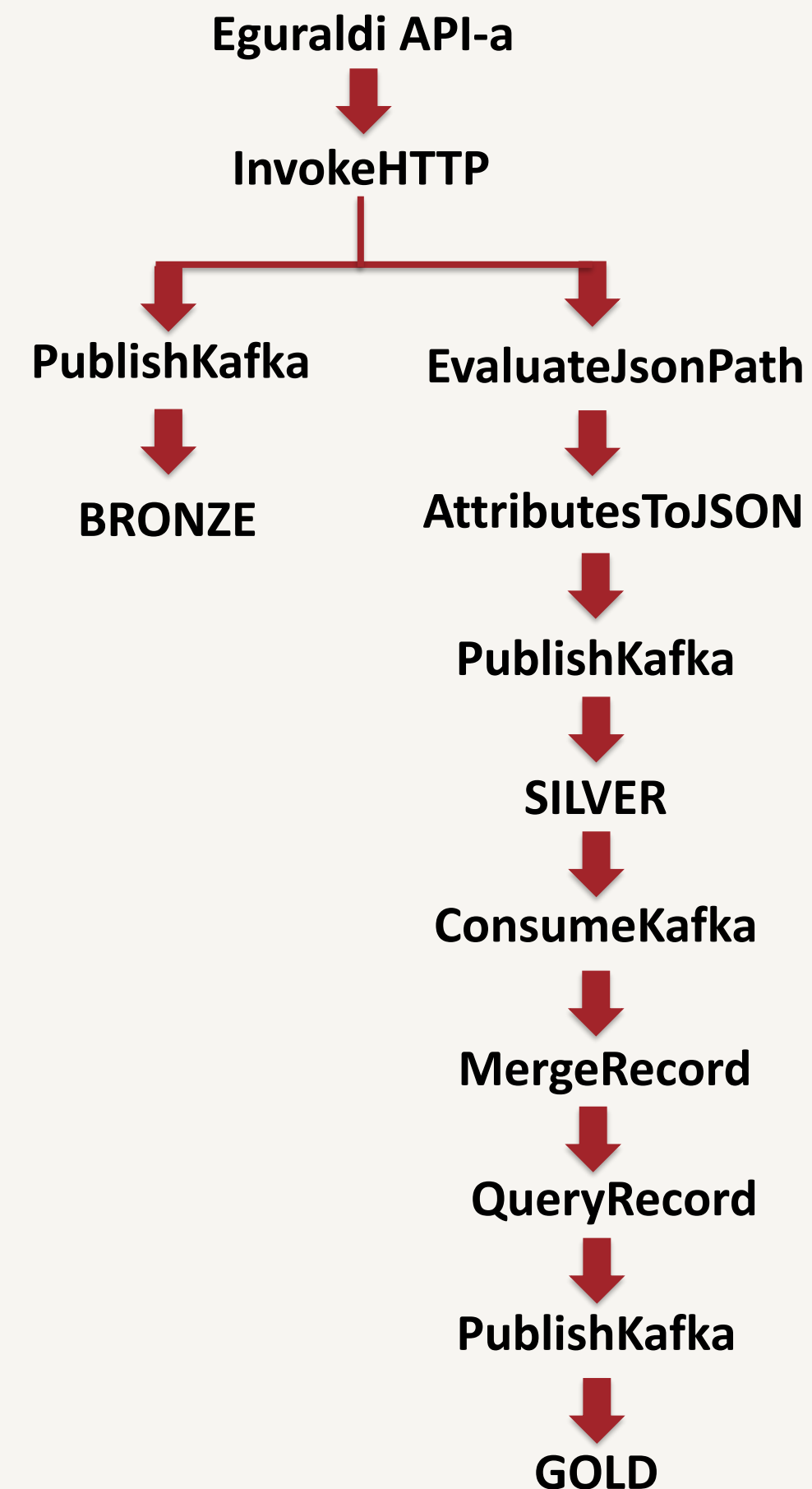
— ZER EGINGO DUGU?

AEMET NiFi-rekin

HELBURUA: aurreko kasuko datu-pipeline bera beste tresna batekin eraikitzea.

- **Bronze** – datu gordina
- **Silver** – datu garbia
- **Gold** – datu agregatua

Ç



REST API → Bronze

- **Sortu fluxua:** InvokeHTTP → PublishKafka
- **API-a:** <https://www.el-tiempo.net/api/json/v2/provincias/03/municipios/03065>
- **Topic-a:** **iabd-aemet-brontzea**

Pistak:

- **InvokeHTTP:**
HTTP Method: GET
Remote URL: API URL
- **PublishKafka:**
Kafka Connection Service
Topic Name

REST API → Silver

- **InvokeHTTP-tik bigarren adarra sortu:** InvokeHTTP → EvaluateJsonPath → AttributesToJSON → PublishKafka
- **Silver-en behar ditugun eremuak:**
 - hiria
 - Temperatura
 - hezetasuna

Pistak:

- **EvaluateJsonPath:**
 - hiria: \$.municipio.NOMBRE
 - temperatura: \$.temperatura_actual
 - Hezetasuna: \$.humedad
 - Destination : flowfile-attribute
- **AttributesToJSON:**
 - Attributes List: hiria,temperatura,hezetasuna
 - Destination : flowfile-content

Silver Kafka-tik kontsumitu

- **Osatu:** AttributesToJSON → PublishKafka → **iabd-aemet-zilarra**
- **Egiaztatu bi topic-ak:**
 - **iabd-aemet-brontzea** → JSON osoa
 - **iabd-aemet-zilarra** → hiria + tenperatura + hezetasuna
- **Beste Process Group batean:** **iabd-aemet-zilarra** → ConsumeKafka

Pistak:

- **ConsumeKafka:**
Kafka Connection Service
Topics: iabd-aemet-zilarra
Group ID: iabd-4kasua-gold

10 neurketa bildu

- **10 Silver neurketa FlowFile bakarrean bildu: ConsumeKafka → MergeRecord**
 - Record Reader → JsonTreeReader
 - Record Writer → JsonRecordSetWriter
 - Minimum Number of Records → 10
 - Maximum Number of Records → 10
- **Gold agregazioa: MergeRecord → QueryRecord**

Kalkulatu SQL-an:

 - batez_besteko_temperatura
 - temperatura_min
 - temperatura_max
 - batez_besteko_hezetasuna

Gold Kafka-ra bidali

- Osatu fluxua:

ConsumeKafka



MergeRecord



QueryRecord



PublishKafka



iabd-aemet-urreea

- Egiaztatu:

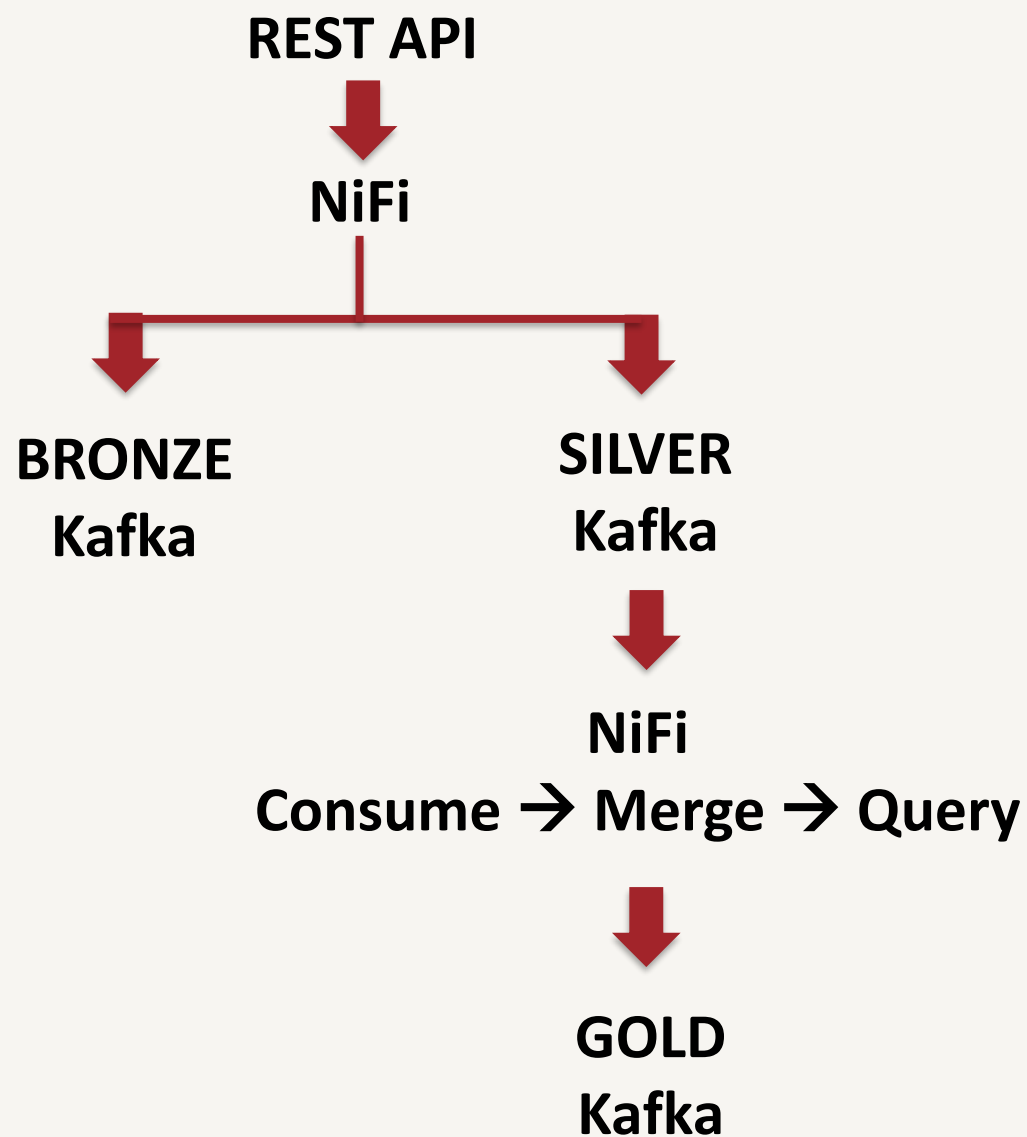
kafka-console-consumer.sh --topic iabd-aemet-urreea...

- Espero den antzeko emaitza:

```
{  
  "hiria": "Elche/Elx",  
  "batez_besteko_tenperatura": 20.8,  
  "tenperatura_min": 19.0,  
  "tenperatura_max": 22.0,  
  "batez_besteko_hezetasuna": 57.4  
}
```

Azken egiaztapena eta hausnarketa

- Amaierako pipeline-a:



- Hausnarketa:

1. Zer egiten du InvokeHTTP-k?
2. Zer desberdintasun dago Bronze eta Silver artean?
3. Zertarako erabiltzen dugu MergeRecord?
4. Zer egiten du QueryRecord-ek?
5. Zer baliokide dira 3. kasuko Pandas kalkuluak 4. kasuan?
6. Zer desberdintasun ikusten duzu Python eta NiFi bidezko implementazioen artean?

04 Kafka Connect

Kafka Connect

Kafka Connect

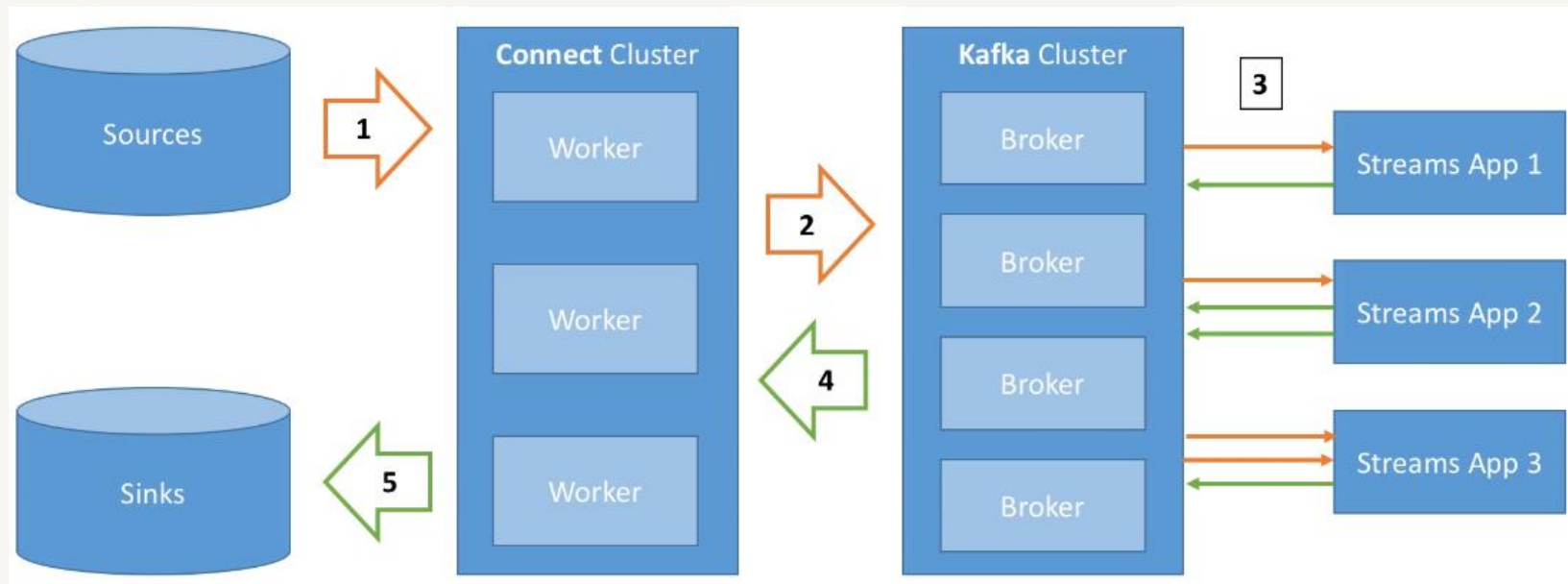
Kafka-n datuak INPORTATZEKO eta ESPOINTATZEKO tresna, ITURRI-KONEKTORE (source) eta HELMUGA-KONEKTORE (sink) bidez. KODE IDATZI GABE datuak sistema ezberdinetatik Kafka-ra eta Kafka-tik kanpora transferitzen ditu.

KODE GABE

Konfigurazio-fitxategi-bidez (.properties) soilik — programatzeko-beharrik-gabe. 100+ konektore ofizial: MySQL, MongoDB, S3, Elasticsearch, Salesforce, etab.

— ARKITEKTURA

Kafka Connect — source eta sink konektoreak



- EZKER ALDEAN: source-systems (MySQL, MongoDB, FTP, S3, REST APIs...)
- **SOURCE CONNECTORS** — source-systems-tik Kafka-topic-etara
- **KAFKA** — erdiko mezularitza-hub (topic-etan biltzen)
- **SINK CONNECTORS** — Kafka-topic-etatik downstream-systems-etara
- ESKUIN ALDEAN: sink-systems (Elasticsearch, HDFS, S3, MongoDB, BigQuery...)

MySQL categories tauladunaren egitura

categories-schema.sql

sql

```
1 describe categories;
2
3 -- Field                | Type          | Null | Key
4 -- category_id          | int(11)       | NO   | PRI auto_increment
5 -- category_department_id | int(11)       | NO   |
6 -- category_name        | varchar(45)   | NO   |
7
8 -- HELBURUA:
9 --   Tauladun honen errenkada-guztiak Kafka-topic-batera
10 --   ESPLIZITUKI-PROGRAMATU-GABE — Kafka Connect-ek
11 --   automatikoki egiten du.
12 --
13 -- RETAIL_DB datu-baseko tauladun-bat
14 --   (DataFlow 2-ko 6. kasuan ere erabili genuen)
15 --
16 -- MODE=incrementing
17 --   incrementing.column.name=category_id
18 --   -> Connect-ek category_id-aren-arabera detektatzen
19 --   incrementing.column.name=category_id
```

connect-standalone.properties — worker-config

```
●●● connect-standalone.properties properties

1 # Kafka-broker-en bootstrap (Connect-ek mezuak hor bidaltzen)
2 bootstrap.servers=localhost:9092
3
4 # Key eta Value converter-ak (JSON estandarra)
5 key.converter=org.apache.kafka.connect.json.JsonConverter
6 value.converter=org.apache.kafka.connect.json.JsonConverter
7
8 # Eskema-informazioa mezuetan barneratu (schema + payload)
9 key.converter.schemas.enable=true
10 value.converter.schemas.enable=true
11
12 # Offset-jarraipena fitxategi-batean (standalone mode)
13 offset.storage.file.filename=/tmp/connect.offsets
14
15 # JAR-konektoreen kokapena (deskargatu eta hor ipini)
```

mysql-source-connector.properties

```
●●● mysql-source.properties properties

1 # Konektore-izena (kluster-mailan UNIKOA)
2 name=mysql-source
3
4 # Konektore-klasea - Confluent JDBC source
5 connector.class=io.confluent.connect.jdbc.JdbcSourceConnector
6
7 # Task-paralelismoa (taula-bakoitzeko-task; gehienez taula-kop)
8 tasks.max=1
9
10 # MySQL-konexioa (retail_db datu-basea)
11 connection.url=jdbc:mysql://localhost/retail_db
12 connection.user=iabd
13 connection.password=iabd
14
15 # Zein taula irakurri (whitelist edo blacklist)
16 table.whitelist=categories
17
18 # Detekzio-modua: incrementing (auto-increment-id-arabera)
19 # incremental.mode=incrementing
```

Aurrebaldintzak — plugin eta driver-deskarga

- CONFLUENT JDBC KONEKTOREA deskargatu → /opt/kafka_*/plugins/
- MYSQL DRIVER (mysql-connector-j-X.jar) → /opt/kafka_*/lib/ edo plugin-direktorioan
- **GUAVA LIBURUTEGIA (necesario zenbait bertsiotan)** — ClassPath-ean
- PLUGIN.PATH konfigurazioa → JAR-aurkitze-direktorioa (recursive)
- **CONFLUENT-HUB CLI alternatiba** — 'confluent-hub install <plugin>' komandoarekin automatikoa

Connect abiarazi + topic-a kontsumitu

```
connect-run.sh bash

1 # Connect-standalone abiarazi (worker + konektorea aldi berean)
2 connect-standalone.sh \
3   config/connect-standalone.properties \
4   config/mysql-source-connector.properties
5
6 # TERMINAL ALDI BEREAN - Topic-emaitza kontsumitu
7 kafka-console-consumer.sh \
8   --topic iabd-retail_db-categories \
9   --from-beginning \
10  --bootstrap-server iabd-virtualbox:9092
11
12 # OHARRAK:
13 #   connect-standalone.sh -> worker-process abiarazten
14 #   2. argumentua -> konektore-konfigurazioa
15 #   Topic-izena -> 'iabd-retail_db-' + 'categories'
```

Topic-ean agertzen den JSON-egitura

```
... output.json json

1 {
2   "schema": {
3     "type": "struct",
4     "fields": [
5       {"type": "int32", "optional": false, "field": "category_id"},
6       {"type": "int32", "optional": false, "field": "category_department_id"},
7       {"type": "string", "optional": false, "field": "category_name"}
8     ],
9     "optional": false,
10    "name": "categories"
11  },
12  "payload": {
13    "category_id": 1,
14    "category_department_id": 2,
15    "category_name": "Football"
16  }
17 }
```

Kafka Connect REST APIa — 8083 port-ea

```
connect-rest.sh bash

1 # Connect-aren version eta kluster-info
2 curl http://iabd-virtualbox:8083/
3
4 # Eraitza:
5 # {"version":"3.3.1",
6 #   "commit":"839b886f9b732b15",
7 #   "kafka_cluster_id":"iHa0JUnTSfm85fvFadsylA"}
8
9 # Konektore aktiboak zerrendatu
10 curl http://iabd-virtualbox:8083/connectors
11
12 # Eraitza:
13 # ["mysql-source-connector"]
14
15 # Konektore-zehatzaren status-a
16 curl http://iabd-virtualbox:8083/connectors/mysql-source/status
17
18 # Konektore-deploy berria (POST + JSON-body)
```

MySQL → Kafka → MongoDB bide osoa

```
connect-both.sh

1 # Source + Sink konektoreak ALDI BEREAN abiarazi
2 # Standalone-mode-an, argumentu-anitz worker-bakar-batean
3
4 connect-standalone.sh \
5   config/connect-standalone.properties \
6   config/mysql-source-connector.properties \
7   config/mongodb-sink-connector.properties
8
9 # EMAITZA:
10 #   1. mysql-source -> MySQL-categories taula-tik mezuak
11 #       'iabd-retail_db-categories' topic-era
12 #   2. mongodb-sink -> 'iabd-retail_db-categories' topic-etik
13 #       mezuak MongoDB-bilduma-batean idatzi
14 #
15 #   MYSQL-EAN insert-egitean -> AUTOMATIKO MONGODB-EAN agertu
16 #   (Connect-ek bide osoa kudeatzen - kode idatzi gabe!)
17 #
18 #   DISTRIBUTED-MODE-AN hobeagoa fault-tolerance-rako
19 #   (DEFACTO bide bakarra, baina hobeagoa da)
```

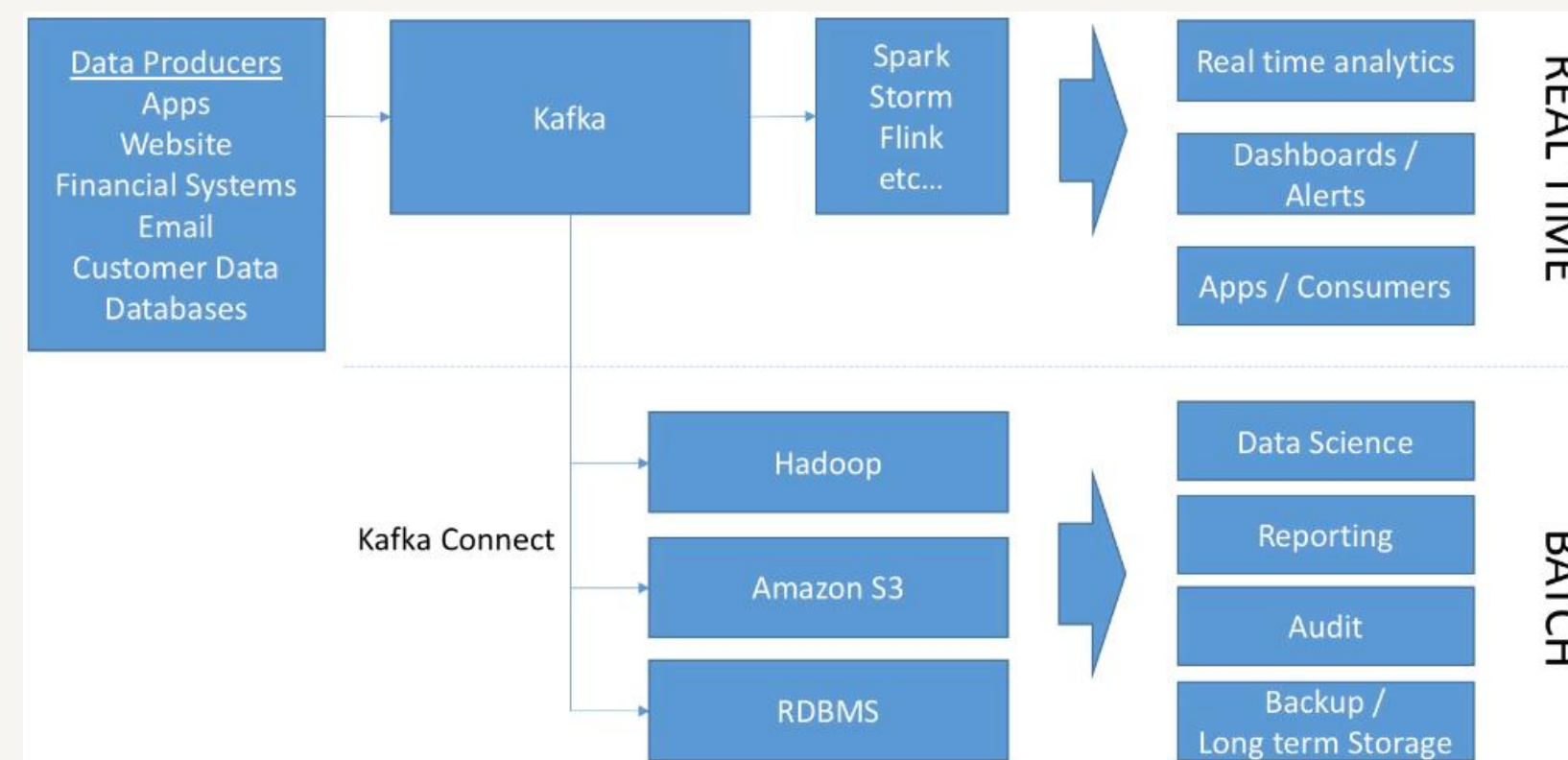
05

Kafka eta Big Data

— KAPPA ARKITEKTURA

Kafka Big Data arkitekturan — batch + streaming

- **KAFKA = MEZULARITZA-HUB ZENTRALA** — datu-iturri-anitzentzat
- **BATCH-INGEST eta STREAMING-INGEST BATERATUTA** — Kafka-bidez bietarako bide-bera
- **KAPPA ARKITEKTURA** — Lambda-aren-alternativa: STREAMING-FIRST, batch-bidearen-beharrik-gabe
- **BIG-DATA-EKOSISTEMA:** Kafka + Spark/Flink + data-lake (S3/HDFS) + DW (Snowflake/BigQuery)
- Kafka Connect = inbiltze-PUNTU NAGUSIA datu-iturri-anitzentzat



Gogoan hartu beharrekoak

-
- i* **Kafka klusterra** — 3 broker minimum produkzioan, broker-ID + port + log-dirs UNIKO-anitz konfigurazio-fitxategi-bana.

 - ii* **Partition + RF** — KLUSTERRA SORTU AURRETIK pentsatu (aldaketak ordena/errendimendua kaltetzen).

 - iii* **Medallion Kafka-en** — Bronze/Silver/Gold TOPIC-INDEPENDENTEAK; pipeline-modularizazioa + consumer-anitzen-aukera.

 - iv* **NiFi + Kafka** — PublishKafka + ConsumeKafka prozesadoreak; record-aldaerak schema-aware produkzio-best-practice.

 - v* **Kafka Connect** — kode-idatzi-gabe datu-iturri-anitz integrazioa; produkzio-ohikoa CDC-rako Debezium-ekin.

 - vi* **Kappa arkitektura** — STREAMING-FIRST eredua; Kafka + Spark/Flink + data-lake produkzio-estandar modernoa.

Galderak, zalantzak, iruzkinak.

Hurrengo saioa: DataFlow 5 — Spark Streaming.

APACHE KAFKA	kafka.apache.org/documentation/
CONNECT	#connect
DOKUMENTAZIOA	
CONFLUENT HUB —	hub.confluent.io
KONEKTORE-	
KATALOGO	
DEBEZIOUM CDC	debezium.io
PLATFORMA	
KAPPA	kappa-architecture.com
ARCHITECTURE	
(JAY KREPS)	
JATORRIZKO	Aitor Medrano · CC BY-NC-SA
MATERIALA	